Lecture 9
I/O Devices

Introduction to Computer Systems

Instructors: Wei Xu, Mingyu Gao

Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Spring 2026

IIIS, Tsinghua University

Contents

1 Introduction **1**

2 I/O Devices in a Computer System **1**
2.1 The I/O Path: Buses, Bridges, and Speed Domains . . . 1
2.2 Diversity of Device Behavior . . . 2
2.3 Granularity, Access Pattern, and Timing . . . 2

3 Memory-Mapped I/O **3**
3.1 Registers, Buffers, and I/O Addresses . . . 3
3.2 Protection Through Virtual Memory . . . 4
3.2.1 System calls . . . 5
3.2.2 Mapping I/O into virtual memory . . . 5

4 Standardizing I/O Interfaces **6**
4.1 Why Device Drivers Exist . . . 6
4.2 Block, Stream, Character, and Network Devices . . . 7
4.2.1 Block devices . . . 7
4.2.2 Stream and character devices . . . 7
4.2.3 Network devices . . . 7
4.3 Timing Interfaces: Blocking, Non-blocking, and Asynchronous . . . 7

5 I/O Notification: Polling and Interrupts **8**
5.1 The Notification Problem . . . 8
5.2 Polling . . . 8
5.3 Interrupts . . . 9
5.4 Comparing Polling and Interrupts . . . 10

6 Programmed I/O and Direct Memory Access **10**
6.1 Programmed I/O . . . 10
6.2 DMA as a Data Movement Engine . . . 10
6.3 Why DMA Completion Usually Uses Interrupts . . . 12

7 DMA Design Issues **12**
7.1 Virtual Memory and Memory Pinning . . . 12
7.2 Cache Coherence . . . 12
7.2.1 Software-managed coherence . . . 13
7.2.2 Hardware-coherent I/O . . . 13

8 Magnetic Hard Disks **13**
8.1 Physical Structure . . . 13
8.2 Disk Access Time . . . 14
8.3 Performance Example . . . 16
8.4 Locality and Caching . . . 16

9 Disk Scheduling **16**
9.1 The Scheduling Problem . . . 16
9.2 FIFO . . . 17
9.3 Shortest Seek Time First . . . 17
9.4 SCAN and C-SCAN . . . 18

10 Solid State Drives **18**
10.1 Architecture and Advantages . . . 18
10.2 Pages, Blocks, Reads, Writes, and Erases . . . 19
10.3 Out-of-Place Updates and Garbage Collection . . . 20
10.4 Flash Translation Layer . . . 20
10.5 Wear Leveling . . . 21
10.6 SSD Performance Summary . . . 22

11 GPUs as I/O Devices and Accelerators **22**
11.1 GPU Placement in the System . . . 22
11.2 Communication Overhead and Overlap . . . 23
11.3 CPU-GPU Roles . . . 24
11.3.1 CPU as manager, GPU as accelerator . . . 24
11.3.2 CPU and GPU as cooperative coprocessors . . . 24

12 Putting the Pieces Together **24**

13 Summary **25**

14 Quizzes & Exercises **26**
14.1 Short-Answer Quiz . . . 26
14.2 Answer Key . . . 26
14.3 Essay Questions . . . 27
14.4 Additional Practice Problems . . . 27
14.5 Glossary of Key Terms . . . 27

1 Introduction

A computer that cannot exchange information with the external world is merely an isolated calculating engine. Processors and memories provide the internal machinery for computation, but input/output (I/O) devices provide the connection between computation and its purpose: receiving commands, ingesting data, storing state, displaying results, communicating with other computers, and interacting with humans. A keyboard, mouse, display, network interface, disk, SSD, camera, speaker, and GPU all appear under the broad label of I/O, even though their physical behavior and performance characteristics are dramatically different.

The central theme of this lecture is that I/O is not just "slow memory." Memory accesses are frequent, regular, and usually served by a carefully engineered memory hierarchy. I/O events are diverse, irregular, and often controlled by physical processes outside the processor's clock domain. Human input arrives at human speed; disk heads move mechanically; network packets appear according to external traffic; SSD writes must obey flash erase constraints; GPUs execute programs on physically separate device memories. These differences force computer systems to introduce specialized interfaces, operating-system abstractions, notification mechanisms, and data movement engines.

Definition

Input/output (I/O) is the set of mechanisms by which a computer communicates with devices outside the processor-memory core. I/O includes human-facing devices, machine-to-machine communication devices, persistent storage devices, and accelerator devices. The operating system mediates most I/O to provide protection, fairness, and uniform abstractions.

Two fundamental challenges motivate the entire design space.

1. **Device diversity.** Systems must support thousands of possible devices whose low-level interfaces differ in registers, commands, data formats, timing, error behavior, and performance. A uniform software interface is needed so that applications do not need to know the register layout of every disk controller or keyboard controller.
2. **Speed and predictability mismatch.** Many devices are much slower than processors and may produce events unpredictably. A processor should not spin wastefully for millions of cycles waiting for a human to type a key or a disk sector to arrive. Conversely, for fast devices, per-byte overhead must be extremely low.

This lecture studies how systems resolve these tensions. We first examine where I/O devices sit in the hardware system, then develop the abstractions of memory-mapped I/O and device drivers. We then discuss notification by polling and interrupts, efficient bulk data movement by direct memory access (DMA), and design issues caused by virtual memory and caches. The second half applies these ideas to disks, SSDs, and GPUs.

2 I/O Devices in a Computer System

2.1 The I/O Path: Buses, Bridges, and Speed Domains

Modern computer systems are built from components operating at very different speeds. The processor core and its caches may run at several gigahertz. DRAM has much higher latency than cache but still operates in a processor-oriented timing domain. Many I/O devices, however, are attached through a separate I/O bus and communicate through controller logic. The path from a device to the CPU therefore usually crosses an **I/O bridge**, which translates between the processor-memory side and the I/O bus side.

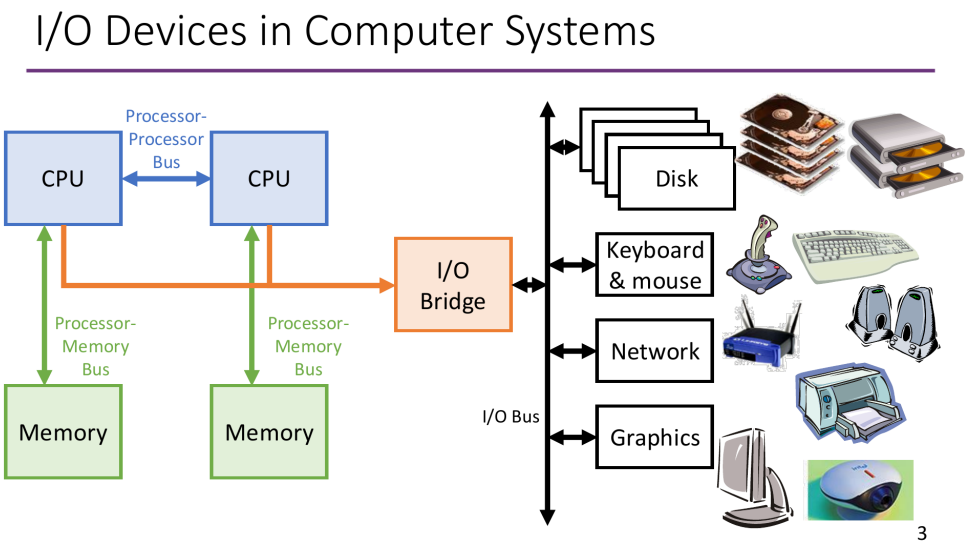


Figure 1: A representative system organization. CPUs and memories communicate through processor-processor and processor-memory buses, while disks, graphics devices, network interfaces, and human-input devices attach through an I/O bus. The I/O bridge connects the slow and diverse I/O side to the faster processor-memory domain.

The bridge is more than a passive wire. It may buffer data, translate bus protocols, arbitrate among devices, forward interrupts, and expose device registers to the processor. On contemporary platforms, examples include PCI Express root complexes, chipset components, and integrated system-on-chip fabrics. Conceptually, the processor sends commands toward device controllers, while device controllers send data, status, and asynchronous notifications back toward the processor.

This division is important because a single access model cannot serve all devices well. A cache line load from DRAM may take tens to hundreds of cycles; waiting for a disk seek may take millions of cycles; waiting for a person to press a key may take billions of cycles. The processor must therefore be decoupled from I/O whenever possible.

2.2 Diversity of Device Behavior

I/O devices can be characterized along several axes: whether they provide input, output, storage, or communication; whether their partner is a human or another machine; their data rate; their response-time requirement; their access granularity; and their access pattern. Table 1 gives representative teaching values for common devices.

Human-facing devices are usually slow in terms of bytes per second but may have important latency requirements: a user notices if mouse motion or keyboard input feels delayed. Machine-facing devices can produce enormous volumes of data. A network card or graphics device may require gigabytes per second of sustained throughput, so even a small amount of CPU overhead per byte can become unacceptable.

2.3 Granularity, Access Pattern, and Timing

A device's **data granularity** is the unit in which it naturally transfers information. A keyboard may deliver one character or key event at a time. A disk or SSD transfers sectors, pages, or blocks. A network interface transfers frames or packets. The natural granularity affects buffering, API design, and the efficiency of DMA.

Table 1: Representative I/O device characteristics. The important observation is not the exact number, but the many-orders-of-magnitude spread between human-speed devices and high-bandwidth devices.

Device	Behavior	Partner	Data rate (MB/s)
Keyboard	Input	Human	0.00001
Mouse	Input	Human	0.02
Laser printer	Output	Human	0.1
Graphics	Output	Human	5000.0
Network	Communication	Machine	1000.0
Magnetic disk	Storage	Machine	600.0
Optical disk	Storage	Machine	10.0

A device's **access pattern** determines whether software can name arbitrary locations. Storage devices are usually random-access: a request can identify a block number, logical sector number, or file offset. Stream devices are sequential: data is read or written as the next byte, character, or packet in a stream. Some devices, such as networks, are sequential from the perspective of received traffic even though their packets contain addresses.

Deep Dive

The phrase *random access* does not mean equal cost. A magnetic disk can seek to any block, but random access is much more expensive than sequential access because the head must move and wait for rotation. An SSD can also read arbitrary pages, and its random-read penalty is much smaller because there are no moving parts. The API may look similar, but the performance model is very different.

The timing interface also matters:

- A **blocking** call waits until the operation has completed or data is available. A process that calls blocking `read()` on a terminal may sleep until the user types input.
- A **non-blocking** call returns quickly with the number of bytes transferred so far, which may be less than requested or even zero. This style is common in event-driven servers.
- An **asynchronous** call initiates work and returns immediately; completion is reported later through a callback, signal, completion queue, future, or other notification mechanism.

The operating system uses these distinctions to schedule computation while slow devices are working.

3 Memory-Mapped I/O

3.1 Registers, Buffers, and I/O Addresses

Every device controller exposes some interface through which software can observe status and issue commands. A simple device might have a status register, a command register, and one or more data registers or buffers. A storage controller may have descriptor queues, doorbell registers, interrupt status registers, error registers, and DMA configuration registers. These registers are determined by hardware vendors and device standards.

Memory-mapped I/O (MMIO) assigns a range of the physical address space to a device. A load from one of these physical addresses reads a device register or buffer; a store to one of

these physical addresses writes a command, data word, or control bit. This gives the processor a familiar instruction-level mechanism for device access, but the target is not DRAM.

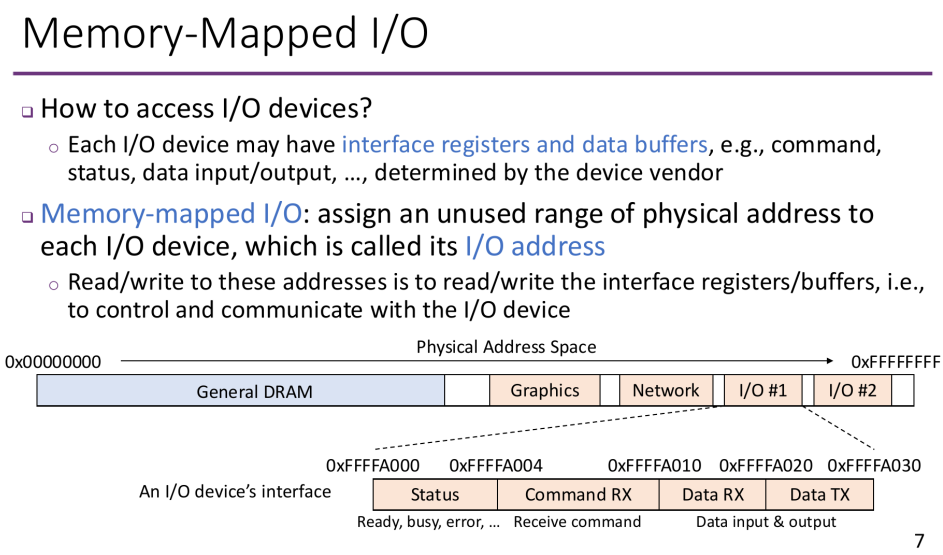


Figure 2: Memory-mapped I/O places device interfaces into unused regions of the physical address space. Stores to a command register can initiate actions; loads from a status register can observe readiness, errors, or completion. Data registers and buffers provide the actual path for input and output.

A simplified register map might look like Table 2.

Table 2: Example memory-mapped device register layout.

Address	Register	Meaning
`0xFFFFA000`	Status	Ready, busy, error flags
`0xFFFFA004`	Command	Start read, start write, reset
`0xFFFFA010`	RX data	Data received from the device
`0xFFFFA020`	RX data	Additional receive buffer
`0xFFFFA030`	TX data	Data to transmit to the device

Because MMIO accesses cause external effects, software must treat them differently from normal cached memory. The compiler must not remove or reorder them as if they were ordinary loads and stores; the hardware must not satisfy them from stale cache lines. Operating systems therefore map MMIO regions with special page attributes, such as uncached, write-combining, strongly ordered, or device-memory semantics depending on the architecture.

3.2 Protection Through Virtual Memory

Raw device access is dangerous. A buggy or malicious user program that can write arbitrary device registers could corrupt the disk, read another process's network packets, overwrite a display buffer, or reprogram DMA to modify kernel memory. Therefore, I/O address ranges are protected by the same virtual memory machinery that protects ordinary memory.

There are two common access paths.

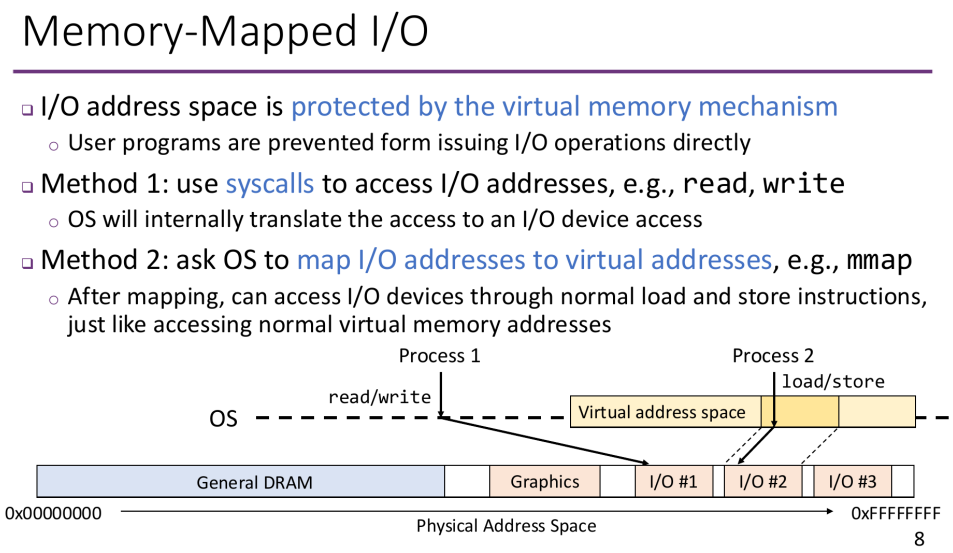


Figure 3: I/O address ranges are protected by virtual memory. A user process normally accesses a device through system calls such as `read()` and `write()`. In special cases, the operating system can map selected I/O pages into a process's virtual address space through mechanisms such as `mmap()`.

3.2.1 System calls

Most applications use system calls such as `open()`, `read()`, `write()`, `ioctl()`, and `close()`. The process crosses into kernel mode; the operating system validates permissions, checks arguments, interacts with the driver, and returns results. The application does not know the device's register addresses.

This path is safe and portable. It also allows the operating system to multiplex devices among processes, buffer data, schedule requests, enforce access control, and recover from errors. The cost is that each system call has overhead: privilege transition, argument validation, possible context switch, and driver work.

3.2.2 Mapping I/O into virtual memory

Some high-performance or specialized software requests that the operating system map a device buffer or register range into the process's virtual address space. The process can then use normal loads and stores to interact with the mapped region. Examples include framebuffer access, memory-mapped files, user-level device drivers in controlled environments, and high-performance networking or GPU runtimes.

Listing 1: A simplified MMIO access pattern in C. Real drivers require architecture-specific barriers and permission checks.

```c
#include <stdint.h>

#define DEV_BASE      0xFFFFA000u
#define REG_STATUS    0x0000u
#define REG_COMMAND   0x0004u
#define REG_TX_DATA   0x0030u
#define STATUS_READY  0x1u
#define CMD_SEND      0x2u

static volatile uint32_t * const dev = (volatile uint32_t *)DEV_BASE;
```

```
11
12 void send_word(uint32_t word) {
13     while ((dev[REG_STATUS / 4] & STATUS_READY) == 0) {
14         /* spin or yield; real code should avoid wasting CPU indefinitely */
15     }
16     dev[REG_TX_DATA / 4] = word;
17     dev[REG_COMMAND / 4] = CMD_SEND;
18 }
```

The `volatile` qualifier tells the compiler that the value can change outside the program's ordinary control flow and that the access must not be optimized away. However, `volatile` is not a complete synchronization solution: many architectures still require memory barriers to enforce ordering between normal memory writes, MMIO writes, and DMA-visible buffers.

Deep Dive

A common systems-programming pitfall is to confuse compiler ordering with hardware ordering. `volatile` affects compiler optimizations, but it does not necessarily flush caches, order writes on a weakly ordered CPU, or make a buffer visible to a DMA engine. Correct driver code often needs explicit memory barriers, cache maintenance instructions, and carefully chosen page attributes.

4 Standardizing I/O Interfaces

4.1 Why Device Drivers Exist

The low-level interface of a device is vendor-specific. One controller might start an operation by setting bit 3 of a command register; another might require writing a descriptor address to a doorbell register; a third might expose a queue in host memory. Applications should not need to know these details. The operating system therefore abstracts devices into a small number of standard types and uses **device drivers** to translate between the abstract interface and hardware-specific actions.

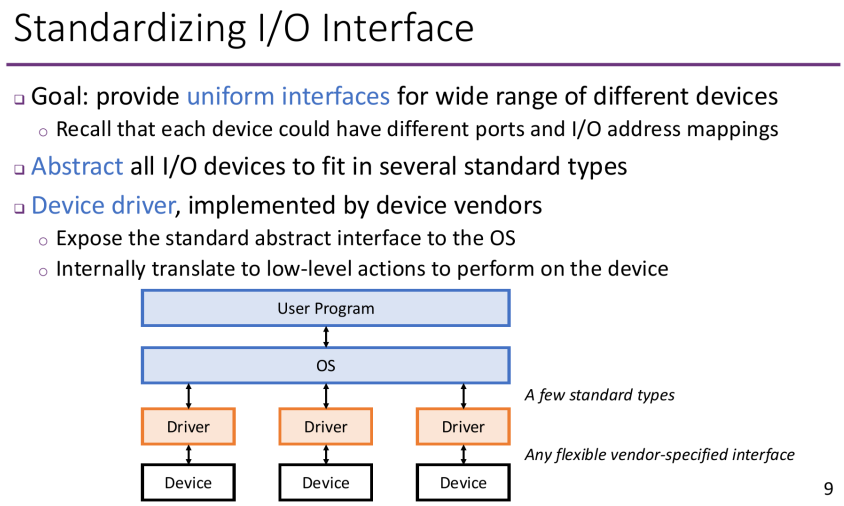


Figure 4: Device drivers expose a small set of standard interfaces to the operating system while hiding flexible vendor-specific register maps, command formats, timing behavior, and error handling.

A driver typically performs the following duties:

- initialize hardware and discover device capabilities;
- expose a standard kernel interface such as a block-device interface, character-device interface, network interface, or GPU runtime interface;
- translate reads, writes, seeks, socket operations, or command submissions into device-specific register accesses and DMA descriptors;
- handle interrupts and errors;
- manage buffers, queues, and synchronization;
- enforce protection and coordinate with virtual memory.

This driver layer is one of the most important examples of abstraction in computer systems. The abstraction is not only for convenience: it also isolates hardware-specific complexity and enables the OS to schedule, protect, and account for device use.

4.2 Block, Stream, Character, and Network Devices

4.2.1 Block devices

A **block device** stores and retrieves fixed-size units of data. Disks and SSDs are the canonical examples. A block device supports commands conceptually similar to:

`open()`, `close()`, `read()`, `write()`, `seek()`

At the kernel level, requests usually name a logical block address and a length. Random access is possible, but, as we will see, mechanical disks and SSDs have very different costs for random access.

4.2.2 Stream and character devices

A **stream device** presents data as an ordered sequence. A keyboard, mouse, serial port, and terminal are often modeled as character or byte devices. The interface can be as simple as `get()` and `put()`, though real systems include buffering, line discipline, echoing, interrupt processing, and device-specific control operations.

A stream device has no meaningful `seek()` operation. Reading "the next byte" is meaningful; asking for byte 17,243 of a live keyboard stream is not.

4.2.3 Network devices

Network interfaces are stream-like but important enough to have specialized APIs. Ethernet, wireless, and Bluetooth devices move packets between machines. Applications usually use sockets, not raw device reads and writes. The network stack interprets packets, enforces protocol rules, multiplexes connections, and manages retransmission or congestion control at higher layers.

4.3 Timing Interfaces: Blocking, Non-blocking, and Asynchronous

The same device can often be accessed through different timing semantics. A blocking disk read may put the process to sleep until the requested block is available. A non-blocking read on a socket may return immediately with fewer bytes than requested. An asynchronous I/O interface may accept a buffer and return before any data has moved, later reporting completion through a completion queue.

Table 3: Comparison of timing interfaces.

Interface	Call behavior	Typical use	Main pitfall
Blocking	Waits until data is ready or operation completes	Simple programs, shell commands, ordinary file reads	Can waste time if a thread must remain responsive
Non-blocking	Returns quickly with partial progress or zero bytes	Event loops, servers, polling multiple sources	Programmers must handle short transfers and retry logic
Asynchronous	Submits request and reports completion later	High-throughput storage, networking, GPU copies	Requires lifetime management of buffers and completion state

Deep Dive

In C and C++, I/O functions may legally transfer fewer bytes than requested. Robust code must loop until the desired amount has been transferred, end-of-file occurs, or an error is reported. Non-blocking I/O adds more cases: a call can return "would block," meaning that the operation should be retried when the device or file descriptor becomes ready.

5 I/O Notification: Polling and Interrupts

5.1 The Notification Problem

An I/O device can become interesting at any time. A packet may arrive, a disk request may complete, an asynchronous write may fail, or a user may press a key. The CPU and operating system must find out without wasting too many cycles and without waiting too long. Two canonical strategies are **polling** and **interrupts**.

5.2 Polling

With polling, the device places information in a status register, and software periodically reads that register. If the status indicates readiness or completion, the OS performs the required action. Otherwise, it may spin briefly, yield the CPU, or sleep for a short time.

Algorithm 1: Polling-Based Device Write

Input: A memory buffer `buf`, length `n`, and device registers `status`, `data`
Output: All bytes requested for transfer are submitted to the device

```
for i ← 0 to n − 1 do
    while device is not ready according to status do
        optionally sleep, yield, or execute a pause instruction;
    write buf[i] to the device data register;
return submitted byte count;
```

Polling is simple and gives the processor explicit control. It can also be efficient when events are frequent and predictable. For example, if a device produces data at a fixed high rate, repeatedly checking a status register may avoid the overhead of taking an interrupt for every small event.

The difficulty is choosing the polling interval. Poll too often and the CPU wastes cycles

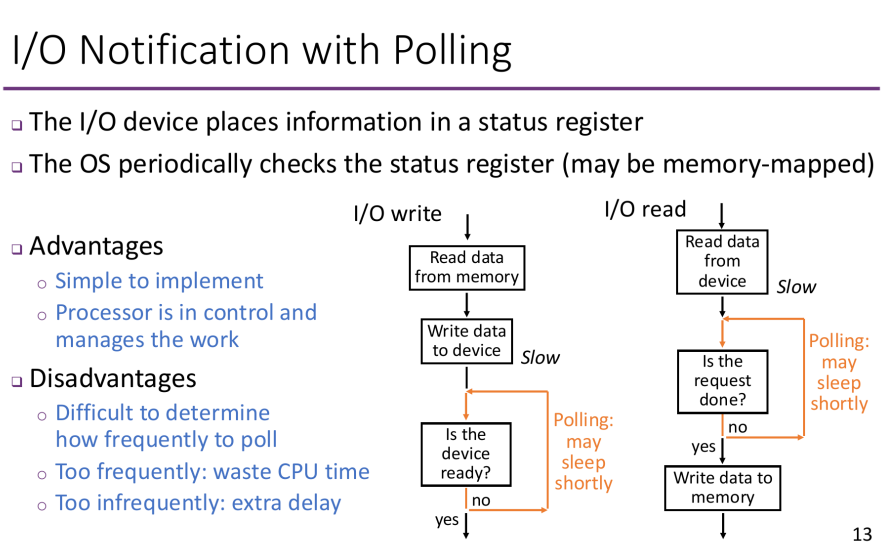


Figure 5: Polling-based I/O. For output, the processor checks whether the device is ready before writing data. For input, it checks whether the request is complete before reading from the device. The frequency of polling determines the trade-off between wasted CPU time and response delay.

reading a register that has not changed. Poll too rarely and the system adds unnecessary latency. This is especially problematic for human input and rare errors, where long idle periods are common.

5.3 Interrupts

An **interrupt** is an asynchronous signal from an external source that causes the processor to stop its current execution path, save state, and enter an interrupt handler. The handler determines the cause, services the device, and returns control to the interrupted computation.

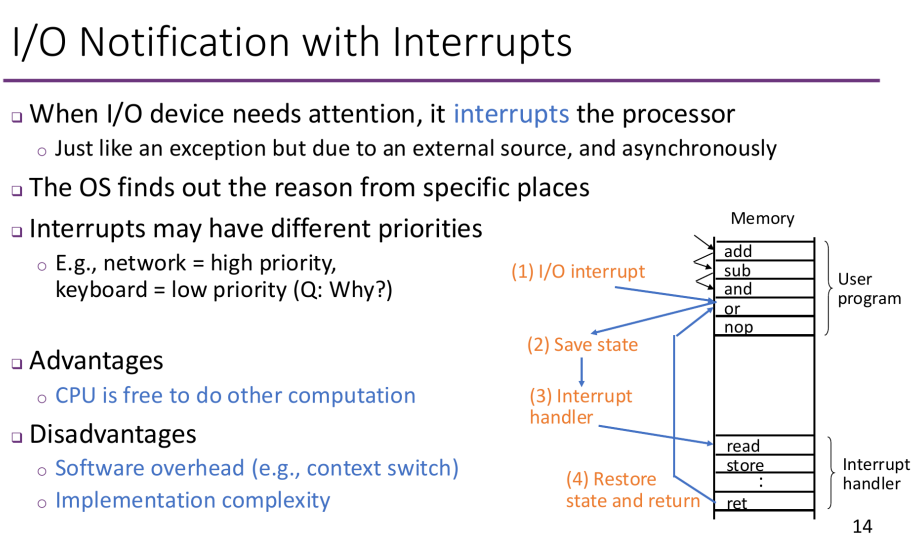


Figure 6: Interrupt-based notification. A device interrupt causes the processor to save state, enter an interrupt handler, service the event, restore state, and resume execution. This avoids continuous polling but introduces software overhead and implementation complexity.

Interrupts are valuable when events are infrequent or unpredictable. The CPU can compute,

run another process, or enter a low-power state until the device asks for attention. This is why keyboard input, disk completion, and many network events are naturally interrupt-driven.

Interrupts also have costs. Entering and exiting a handler requires saving and restoring architectural state. The OS may need to identify the source, acknowledge the device, update kernel data structures, wake sleeping processes, and perhaps perform a context switch. If interrupts arrive too frequently, the system can spend too much time in interrupt handling and too little time doing useful work.

Interrupts may also have priorities. A high-speed network device may receive high priority because delayed service can cause packet buffers to overflow and drop traffic. A keyboard can usually tolerate lower priority because humans type slowly and keyboard controllers often buffer events.

5.4 Comparing Polling and Interrupts

Table 4: Polling versus interrupts.

Mechanism	Best when	Main cost
Polling	Events are frequent, predictable, or so frequent that interrupt overhead would dominate	Wasted CPU time or added latency depending on polling rate
Interrupts	Events are infrequent, unpredictable, initiated by the device, or occur while CPU has other work	Handler overhead, context switching, priority and concurrency complexity

A common hybrid approach is to use interrupts to detect that work has arrived, then poll briefly to drain a batch of events. High-performance network drivers often use such batching: an interrupt wakes the CPU, the driver processes multiple packets by polling a queue, and interrupts may be temporarily disabled or coalesced to prevent interrupt storms. This combines the low idle overhead of interrupts with the low per-event overhead of polling.

6 Programmed I/O and Direct Memory Access

6.1 Programmed I/O

In **programmed I/O (PIO)**, the CPU explicitly moves each byte or word between device registers and memory using loads and stores. PIO has simple hardware and is easy to understand. However, it wastes CPU cycles for large transfers because the CPU becomes a data-copy engine.

For example, to read a disk block using PIO, the CPU might repeatedly read a device data register and store each word into memory. During that time it cannot use those cycles for computation. This is acceptable for tiny transfers or very simple microcontrollers, but it is inefficient for modern storage, networking, and graphics workloads.

6.2 DMA as a Data Movement Engine

Direct memory access (DMA) offloads bulk data movement to a hardware engine. The CPU sets up a transfer by describing the source, destination, length, operation type, and device identity. The DMA engine then moves data between the device and memory without requiring the CPU to copy every byte. When the operation completes, the DMA engine or device typically notifies the CPU, often with an interrupt.

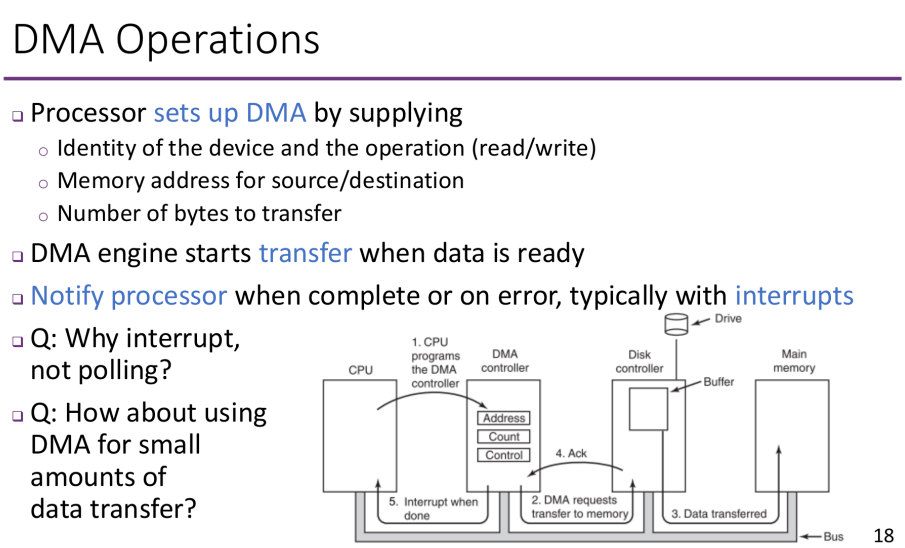


Figure 7: A DMA operation. The CPU programs a DMA controller with the device, memory address, byte count, and control information. The DMA engine performs the transfer over the bus and interrupts the CPU when the transfer completes or fails.

A DMA engine is itself an I/O device with a control interface, often memory-mapped. It may accept descriptors containing fields such as:

- source address;
- destination address;
- number of bytes;
- operation type and flags;
- pointer to the next descriptor;
- completion status.

Multiple descriptors can be placed in a queue. This allows the CPU to batch transfers while the hardware serves them one after another.

Algorithm 2: DMA Transfer Setup and Completion

Input: Device identifier `dev`, operation `op`, memory buffer `buf`, byte count `n`
Output: A completed I/O transfer or an error report

validate that `buf` belongs to the requesting process and has the required permissions;
pin the pages containing `buf` so they cannot be swapped during the transfer;
translate the buffer into one or more physical or I/O-virtual address ranges;
create DMA descriptors containing source, destination, length, and flags;
flush or invalidate caches as required by the direction of transfer;
write the descriptor address to the device or DMA controller doorbell register;
allow the CPU to run other work while the DMA engine transfers data;
if *completion interrupt or completion queue entry arrives* **then**
    check status, unpin pages, wake waiting process, and report result;
return completion status;

DMA is especially effective for large transfers because setup cost is amortized over many bytes. For very small transfers, however, the setup overhead may exceed the savings. A single character from a keyboard is better handled through an interrupt and a small buffer than through a full DMA descriptor setup.

6.3 Why DMA Completion Usually Uses Interrupts

A DMA transfer may take a long and variable time. If the CPU continuously polled for completion, it would waste the very cycles DMA was designed to save. Interrupt-driven completion lets the CPU submit the request and perform useful work until hardware reports that the transfer is done. Polling can still be useful in carefully tuned high-performance paths, especially if completion is expected very soon or if batching many completions reduces overhead.

7 DMA Design Issues

7.1 Virtual Memory and Memory Pinning

DMA exposes a subtle interaction between devices and virtual memory. User programs use virtual addresses, but a traditional DMA engine may use physical addresses. The operating system must translate the user's buffer to physical memory and ensure that those physical pages remain valid while the DMA is pending.

Definition

Memory pinning locks selected virtual memory pages into physical DRAM so that the operating system will not swap them out, move them, or reuse them while an I/O operation is pending. Pinning is necessary when hardware DMA uses physical addresses that must remain stable until completion.

Two issues arise immediately.

First, the OS may otherwise swap pages out to disk. If a DMA engine is told to write into physical page P, and the OS reassigns P to another process before the DMA completes, the device could corrupt unrelated memory. Pinning prevents this.

Second, contiguous virtual addresses are not necessarily contiguous physical addresses. A user buffer from address `0x10000000` to `0x1000FFFF` may span many virtual pages that map to arbitrary physical frames. A DMA engine that only accepts one contiguous physical range cannot directly transfer the whole buffer.

Common solutions include:

- **Bounce buffers.** Copy data through a physically contiguous kernel buffer. This is simple but adds extra copying.
- **Scatter-gather DMA.** Build a list of descriptors, one for each physical segment. The hardware follows the descriptor chain and produces one logical transfer.
- **I/O virtual memory and IOMMU.** Give devices I/O virtual addresses translated by an I/O memory management unit. The IOMMU provides address translation and protection for devices, analogous to an MMU for CPUs.

7.2 Cache Coherence

DMA also interacts with caches. Suppose a device writes new data into DRAM while the CPU still has an older copy of that memory in its cache. If the CPU later reads from the cache, it may observe stale data. Conversely, suppose the CPU has modified a buffer in cache but has not written it back to memory. If a device reads from DRAM, it may read old data instead of the CPU's latest values.

This is another form of cache coherence problem, except the competing agents are CPU caches and I/O devices rather than multiple CPU cores.

There are two broad solution families.

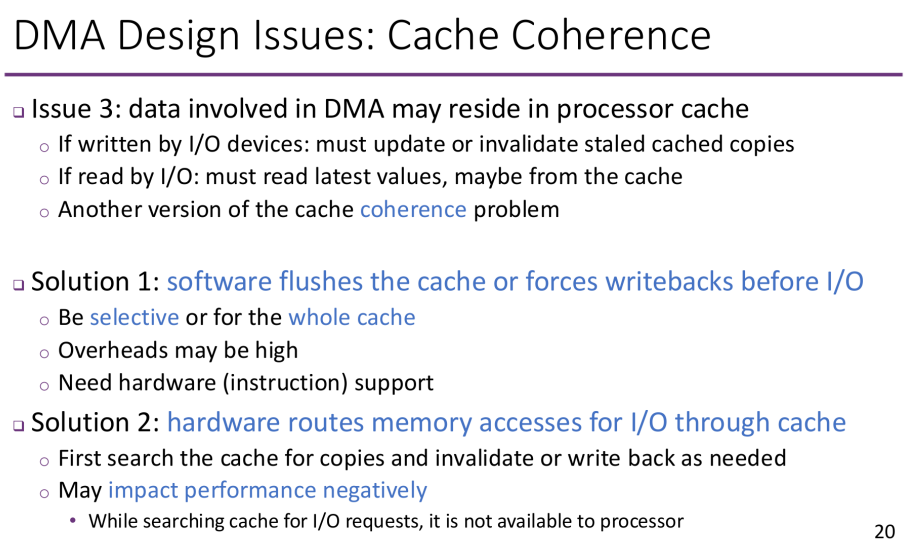


Figure 8: DMA cache-coherence hazards. If a device writes into memory, stale CPU cache lines must be invalidated or updated. If a device reads from memory, dirty CPU cache lines must be written back first. Systems can handle this through software cache maintenance or hardware-coherent I/O paths.

7.2.1 Software-managed coherence

Before a device reads a buffer, the OS or driver writes back dirty cache lines so memory contains the newest data. Before the CPU reads a buffer that a device has written, the OS or driver invalidates stale cache lines. This approach can be selective but requires correct hardware instructions and careful ordering. It is error-prone, and flushing large cache regions can be expensive.

7.2.2 Hardware-coherent I/O

Some systems route I/O memory accesses through coherent interconnects that snoop caches or otherwise maintain coherence. The device or DMA engine can then observe the latest data, and CPU caches can be invalidated or updated automatically. This simplifies software but consumes interconnect bandwidth and may reduce CPU cache availability while I/O requests probe or interact with the cache hierarchy.

Deep Dive

The right coherence strategy is a hardware/software contract. Device drivers must know whether a platform's DMA is coherent. Portable driver frameworks therefore provide functions such as "map buffer for DMA," "unmap buffer," and "synchronize for CPU/device." These calls hide whether the platform implements cache maintenance in software or hardware.

8 Magnetic Hard Disks

8.1 Physical Structure

Magnetic hard disks are long-term, non-volatile storage devices. They provide high capacity at low cost per byte, but they are slow because they rely on mechanical motion. A hard disk

contains one or more rotating **platters**. Each platter has two magnetic surfaces. A moving actuator arm positions read/write heads over tracks on the surfaces.

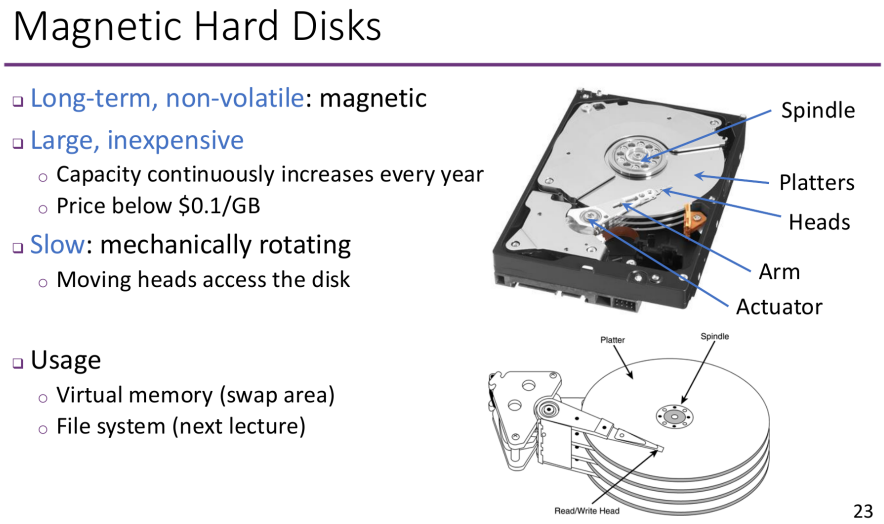


Figure 9: A magnetic hard disk consists of platters rotating around a spindle, an actuator arm, and read/write heads. The mechanical nature of the device explains why random access is much slower than electronic memory access.

Each surface is divided into concentric **tracks**. Each track is divided into **sectors**, often historically 512 bytes or, in modern devices, larger physical sectors such as 4 KiB. Tracks at the same radial position on all surfaces form a **cylinder**. Operating systems typically transfer groups of sectors together as blocks.

Disks are used for file systems and, historically, for virtual-memory swap space. Even when SSDs dominate performance-oriented systems, magnetic disks remain important for capacity-oriented storage.

8.2 Disk Access Time

A disk request has several components:

$$T_{\text{access}} = T_{\text{queue}} + T_{\text{controller}} + T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}. \tag{1}$$

- **Queueing delay** occurs when other requests are waiting or already being served.
- **Controller overhead** is time spent by the disk controller processing the request.
- **Seek time** is the time to move the head assembly to the correct track.
- **Rotational latency** is the time waiting for the desired sector to rotate under the head.
- **Transfer time** is the time for the requested data to pass under the head and be read or written.

For a disk rotating at R revolutions per minute (rpm), one full rotation takes $60/R$ seconds. On average, a random sector requires half a rotation of waiting:

$$T_{\text{rotation,avg}} = \frac{1}{2} \cdot \frac{60}{R} = \frac{30}{R} \text{ seconds}. \tag{2}$$

The key lesson is that small random requests are dominated by seek and rotational latency, not transfer bandwidth.

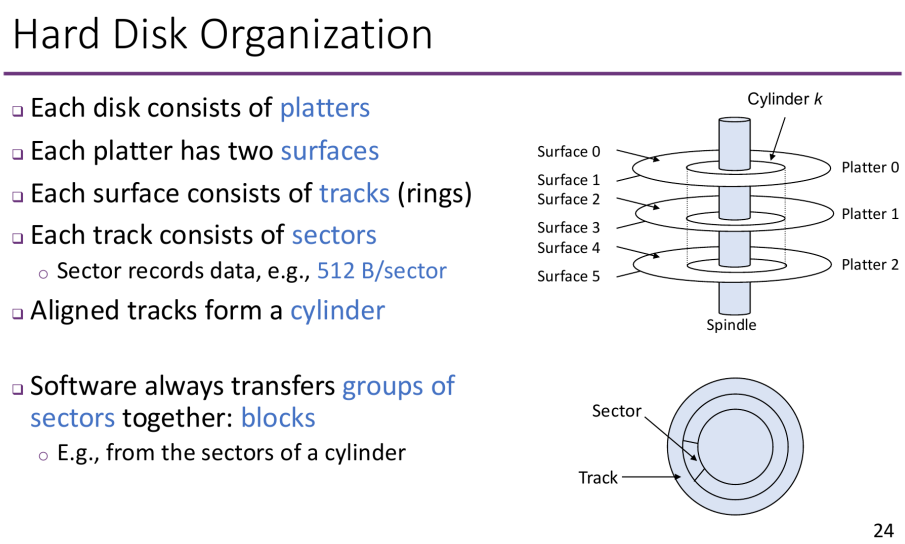


Figure 10: Disk organization. Platters have surfaces; surfaces contain circular tracks; tracks contain sectors; aligned tracks across surfaces form cylinders. Software usually transfers blocks, which are groups of sectors.

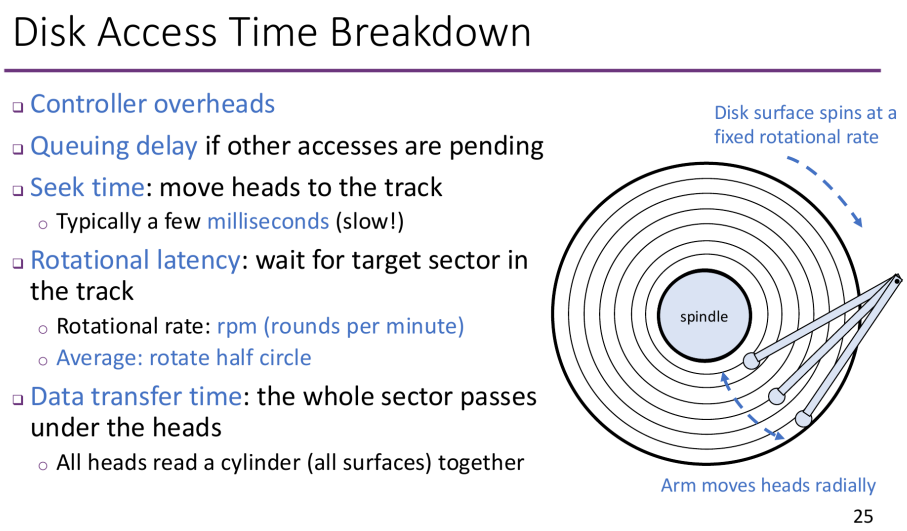


Figure 11: Disk access-time components. The arm moves heads radially to the right track, the platter rotates at a fixed rate until the target sector arrives, and data is transferred as the sector passes under the head.

8.3 Performance Example

Consider a 4 KiB block read from an idle 15,000 rpm disk with 4 ms average seek time, 100 MB/s transfer rate, and 0.2 ms controller overhead. The average rotational latency is:

$$\frac{1}{2} \cdot \frac{1}{15000/60} = 0.002 \text{ s} = 2 \text{ ms}. \tag{3}$$

The transfer time for 4 KiB at 100 MB/s is approximately:

$$\frac{4 \text{ KiB}}{100 \text{ MB/s}} \approx 0.04 \text{ ms}. \tag{4}$$

Thus:

$$T_{\text{read}} \approx 4 \text{ ms} + 2 \text{ ms} + 0.04 \text{ ms} + 0.2 \text{ ms} = 6.24 \text{ ms}. \tag{5}$$

The effective throughput for this small random read is about:

$$\frac{4 \text{ KiB}}{6.24 \text{ ms}} \approx 656 \text{ KiB/s}. \tag{6}$$

Now suppose the system reads 1 MiB sequentially. The seek, rotation, and controller overhead are paid once, while transfer time dominates:

$$T_{\text{read}} \approx 4 \text{ ms} + 2 \text{ ms} + 10 \text{ ms} + 0.2 \text{ ms} = 16.2 \text{ ms}. \tag{7}$$

The throughput becomes roughly 61.7 MB/s, nearly two orders of magnitude better. This example explains why operating systems and file systems work hard to create large sequential disk accesses.

Listing 2: A small Python calculator for the disk-access example.

```python
def disk_read_time_ms(block_kib, rpm, seek_ms, transfer_mb_s, controller_ms):
    rotational_ms = 0.5 * (60.0 / rpm) * 1000.0
    transfer_ms = (block_kib / 1024.0) / transfer_mb_s * 1000.0
    total_ms = seek_ms + rotational_ms + transfer_ms + controller_ms
    throughput_mb_s = (block_kib / 1024.0) / (total_ms / 1000.0)
    return total_ms, throughput_mb_s

print(disk_read_time_ms(4, 15000, 4.0, 100.0, 0.2))
print(disk_read_time_ms(1024, 15000, 4.0, 100.0, 0.2))
```

8.4 Locality and Caching

Disks benefit strongly from locality. Sequential access amortizes seek and rotational costs; spatial locality reduces head movement; temporal locality enables caches to satisfy repeated reads without touching the disk. A disk may have an internal cache, while the operating system also maintains a buffer cache or page cache. File systems exploit locality by placing related blocks near each other and by reading ahead when sequential access is detected.

9 Disk Scheduling

9.1 The Scheduling Problem

When multiple disk requests are queued, the order of service matters. Since seek time often dominates, serving requests in arrival order may produce excessive head movement. A disk

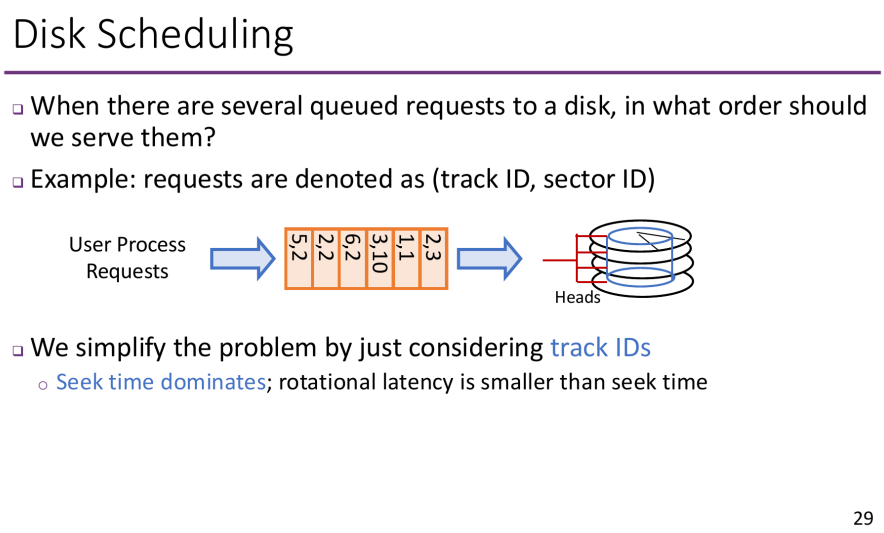


Figure 12: Disk scheduling abstracts requests by track and sector. A simplified analysis often focuses on track IDs because seek time is a dominant component of random-access latency.

scheduler attempts to reduce mechanical movement while preserving fairness and bounded response time.

Suppose the pending track requests are:

$$2,\ 1,\ 3,\ 6,\ 2,\ 5 \tag{8}$$

and the initial head position is track 5.

9.2 FIFO

First-in, first-out (FIFO) serves requests in arrival order:

$$2,\ 1,\ 3,\ 6,\ 2,\ 5. \tag{9}$$

FIFO is fair because old requests are not bypassed. However, it can cause many seeks. In the example, the head moves from 5 to 2, then 1, then 3, then 6, then 2, then 5. This zig-zag movement wastes mechanical time.

9.3 Shortest Seek Time First

Shortest seek time first (SSTF) chooses the request closest to the current head position. In the example, one possible order is:

$$5,\ 6,\ 3,\ 2,\ 2,\ 1. \tag{10}$$

SSTF reduces total head movement and usually improves throughput. Its drawback is starvation: if new nearby requests keep arriving, a far-away request can wait for a long time.

Algorithm 3: Shortest Seek Time First Disk Scheduling

Input: Current head position `head` and a set of pending track requests `Q`
Output: An ordered service sequence

```
sequence ← empty list;
while Q is not empty do
    choose request r in Q minimizing |r.track − head|;
    append r to sequence;
    head ← r.track;
    remove r from Q;
return sequence;
```

9.4 SCAN and C-SCAN

SCAN, also called the elevator algorithm, moves the head in one direction and serves the closest request in that direction. When there are no more requests in that direction, it reverses. The behavior resembles an elevator serving floors as it travels up and down. SCAN reduces seek time and avoids starvation better than SSTF, but it can favor middle tracks because the head passes them more often.

C-SCAN (circular SCAN) serves requests only in one direction. When the head reaches the end, it returns to the beginning without serving requests on the way back. This produces more consistent response times and better fairness across the disk surface, at the cost of longer return seeks and possibly lower throughput.

Table 5: Disk scheduling policies for the example request stream.

Policy	Example order	Trade-off
FIFO	$2, 1, 3, 6, 2, 5$	Fair by arrival time, but many seeks
SSTF	$5, 6, 3, 2, 2, 1$	Low seek overhead, but possible starvation
SCAN	$5, 6, 3, 2, 2, 1$	Low seek overhead and no starvation under stable queues, but favors middle tracks
C-SCAN	$5, 6, 1, 2, 2, 3$	More consistent response time, but longer return movement

On SSDs, mechanical seek time does not exist, so traditional disk scheduling is less important. Still, request ordering matters for other reasons: parallelism across flash channels, write amplification, wear leveling, and queue management.

10 Solid State Drives

10.1 Architecture and Advantages

A solid state drive (SSD) is built from flash memory chips, a controller, and often a DRAM buffer or cache. Unlike magnetic disks, SSDs have no rotating platters or moving heads. As a result, they eliminate seek time and rotational latency, reduce power, improve ruggedness, and provide much faster random access.

An SSD is still much slower than DRAM, but it is far faster than a magnetic disk for small random reads. Typical teaching values in this lecture include read latencies around 20 to 100 μs, compared with milliseconds for disks.

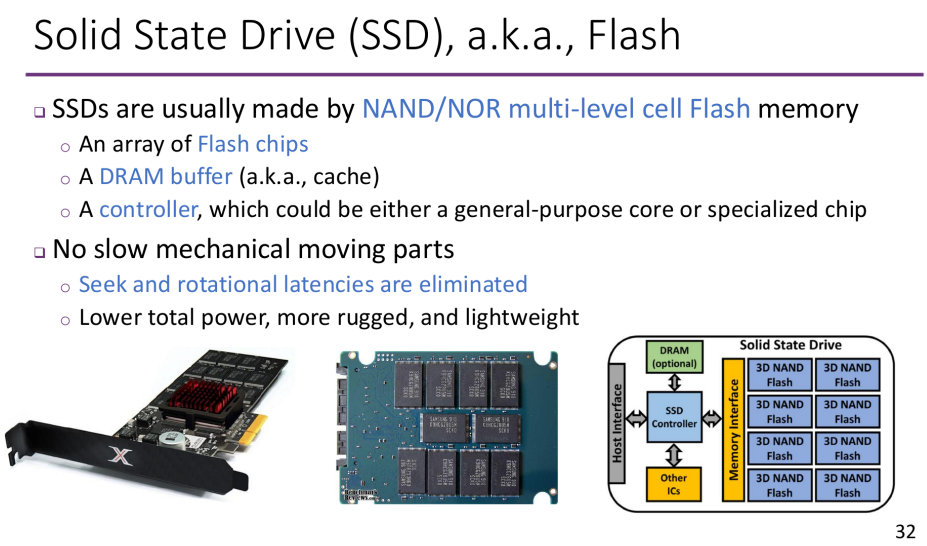


Figure 13: An SSD combines arrays of NAND/NOR flash chips, a controller, and often a DRAM buffer. Removing mechanical motion eliminates seek and rotational latency.

10.2 Pages, Blocks, Reads, Writes, and Erases

Flash memory is organized into pages and blocks. A representative organization is 4 KiB per page and 32 to 128 pages per block. Reads occur in page units and can be relatively fast. Writes are more complicated: flash can write only to erased pages, and erase operations happen at block granularity. A page write might take hundreds of microseconds, while a block erase may take milliseconds.

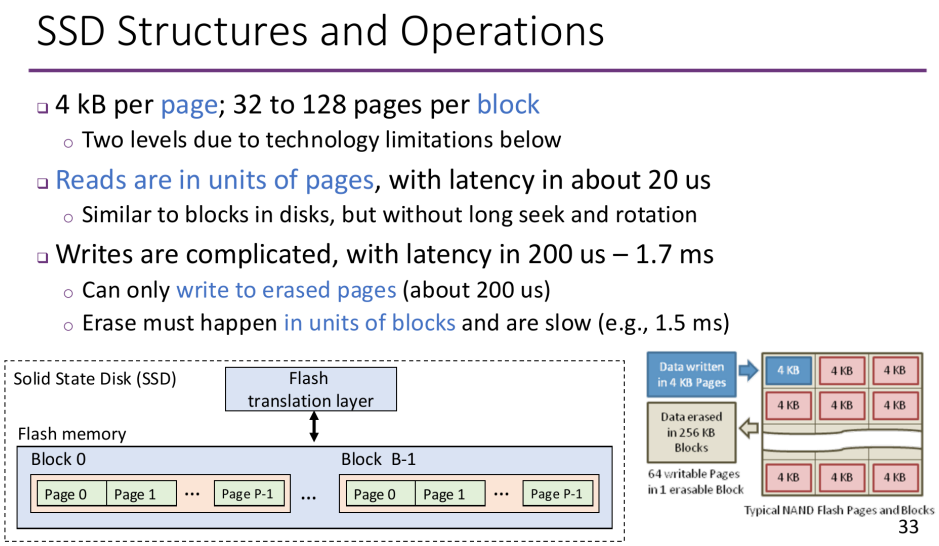


Figure 14: SSD page/block organization. Reads occur in page units. Writes require erased pages, and erase operations happen at the larger block granularity, creating the need for out-of-place updates and controller-level management.

This asymmetry is the root of many SSD performance properties. The drive cannot simply overwrite a page in place the way a magnetic disk can overwrite a sector. Instead, it writes a new version elsewhere and updates its address mapping.

10.3 Out-of-Place Updates and Garbage Collection

Suppose a block contains pages A, B, C, and D. Later, the system modifies those logical pages. The SSD writes new pages A', B', C', and D' into free physical pages and changes the mapping so the logical addresses now point to the new physical locations. The old pages become obsolete. Eventually, the controller must reclaim blocks by copying still-valid pages elsewhere and erasing a block that contains obsolete pages.

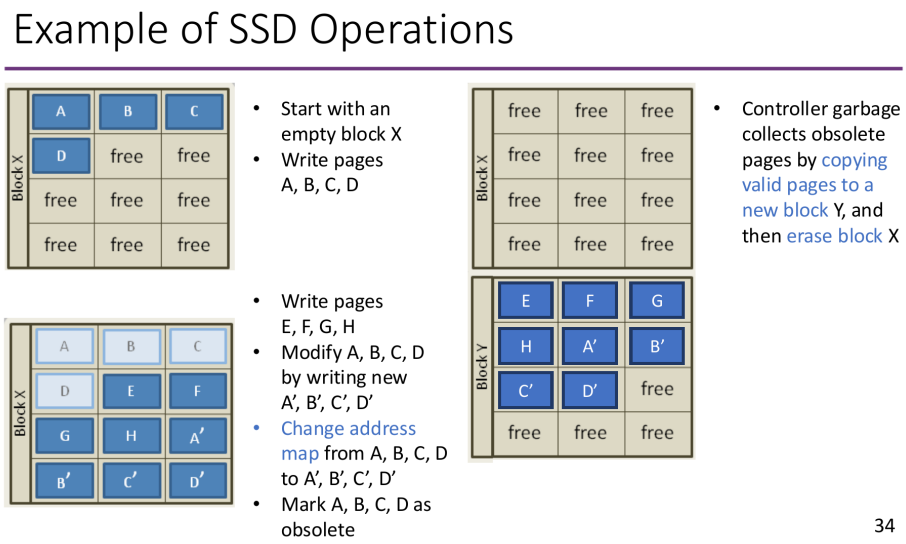


Figure 15: An SSD update sequence. Modified data is written to new pages, the logical-to-physical mapping is changed, old pages are marked obsolete, and garbage collection later copies valid pages to another block before erasing the old block.

This process introduces **write amplification**: writing one logical page may eventually require additional internal reads, writes, and erases. Garbage collection can also create latency spikes if a user write arrives when the drive has little free erased space.

10.4 Flash Translation Layer

The **Flash Translation Layer (FTL)** is the controller software/firmware that manages flash constraints while presenting the host with a block-device interface. The host may believe it is writing logical block 1000 repeatedly; internally, the FTL remaps each new version to a different physical page.

Core FTL functions include:

- logical-to-physical address mapping;
- tracking block state: erased, free, valid, obsolete, bad;
- garbage collection;
- wear leveling;
- bad block management;
- performance scheduling across flash chips and channels.

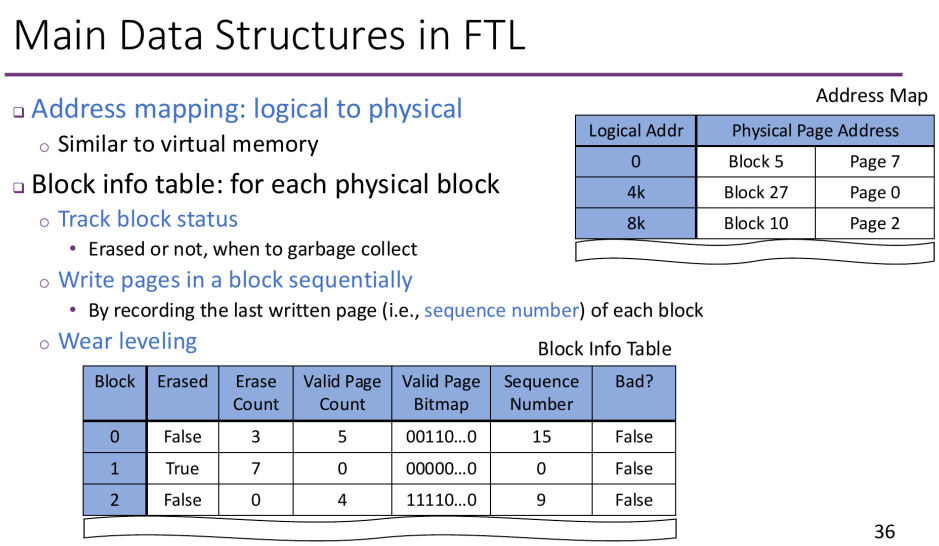


Figure 16: FTL data structures. An address map translates logical addresses to physical pages. A block information table tracks erased status, erase counts, valid-page counts, valid-page bitmaps, sequence numbers, and bad-block status.

Algorithm 4: Out-of-Place Flash Page Write in an FTL

Input: Logical page address `L` and new data `D`
Output: Updated mapping from `L` to a fresh physical page

choose an erased physical page `Pnew` from a block selected by the FTL policy;
program `D` into `Pnew`;
if *`L` was previously mapped to `Pold`* **then**
    mark `Pold` obsolete in the block information table;
    decrement the valid-page count of `Pold`'s block;
update the logical-to-physical map so `L` points to `Pnew`;
increment the valid-page count and sequence number of `Pnew`'s block;
if *free erased space is low* **then**
    select a victim block, copy valid pages elsewhere, and erase the victim block;
return updated mapping entry;

The analogy with virtual memory is useful: both systems maintain mappings from one address space to another. But the goals differ. Virtual memory maps virtual pages to physical frames for protection, sharing, and memory management. The FTL maps logical block addresses to physical flash pages to hide erase-before-write constraints and wear-out behavior.

10.5 Wear Leveling

Flash cells wear out after a finite number of program/erase cycles. A block may tolerate on the order of many thousands to hundreds of thousands of erases depending on technology. Writes and erases require high voltages that gradually degrade cells. If the same physical block is rewritten frequently, it will fail earlier than less-used blocks.

Wear leveling spreads erases across physical blocks so that no small set of blocks becomes a hotspot. Dynamic wear leveling chooses new write locations to avoid heavily used blocks. Static wear leveling may occasionally move cold data from lightly worn blocks so those blocks can also participate in the erase pool.

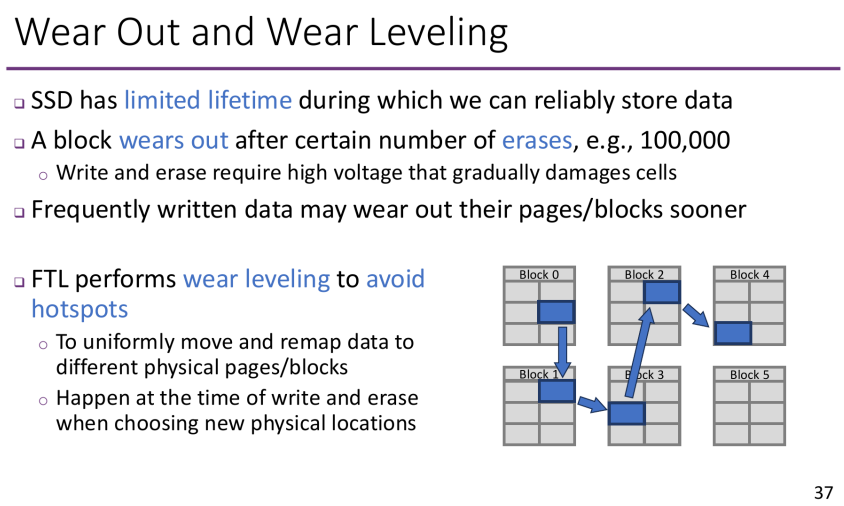


Figure 17: Wear leveling remaps logical data across physical pages and blocks to avoid repeatedly erasing the same locations. The goal is to distribute wear and extend device lifetime.

10.6 SSD Performance Summary

Compared with magnetic disks, SSDs have several advantages:

- no moving parts, which improves latency, power, ruggedness, and weight;
- read latency in tens of microseconds rather than milliseconds;
- much higher random-access performance;
- bandwidth often several times higher than disks, though still far below DRAM;
- random access only slightly slower than sequential access for reads.

Their disadvantages follow from flash technology:

- writes are much more expensive than reads;
- erases are even more expensive and happen at block granularity;
- out-of-place updates create garbage collection and write amplification;
- cells wear out, requiring wear leveling;
- cost per byte is typically higher than magnetic disks.

11 GPUs as I/O Devices and Accelerators

11.1 GPU Placement in the System

A GPU can be viewed as an I/O device attached to the CPU through an I/O interconnect such as PCIe. The CPU and its memory are the **host**; the GPU and its memory are the **device**. Although GPUs are powerful processors, the system-level interaction resembles I/O: software sends commands, copies data, launches kernels, waits for completion, and retrieves results.

A typical offload sequence is:

1. allocate memory on the GPU;
2. copy input data from host memory to device memory;
3. load or reference a GPU program, often called a kernel;
4. execute the kernel on the GPU;

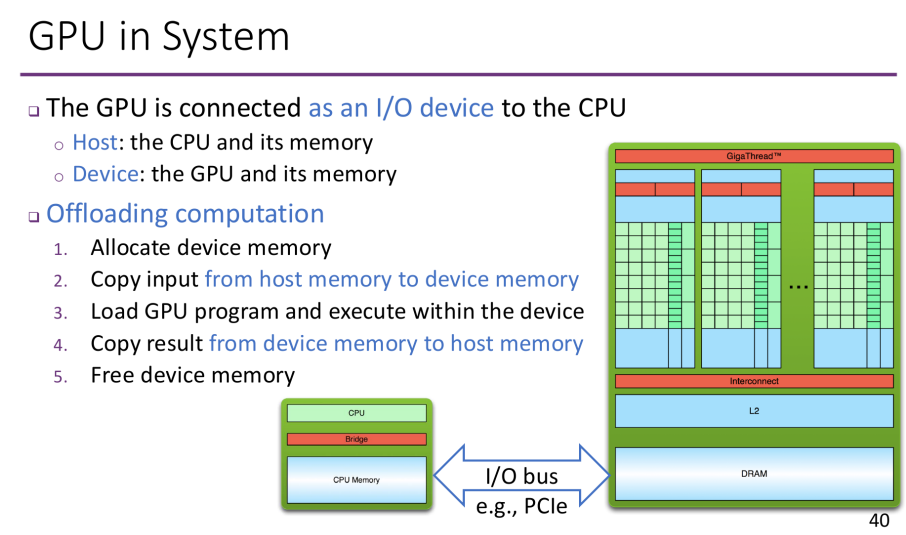


Figure 18: A GPU in the system. The CPU-side host memory and GPU-side device memory are physically separate in many systems. Offloading computation requires allocating device memory, copying inputs, executing GPU code, copying results back, and freeing device memory.

5. copy result data from device memory to host memory;
6. free device memory.

11.2 Communication Overhead and Overlap

The separation between CPU memory and GPU memory creates data movement overhead. A PCIe transfer may have hundreds to thousands of cycles of latency and much lower bandwidth than on-chip or local device memory bandwidth. Therefore, GPU acceleration is profitable only when computation saved exceeds data movement cost.

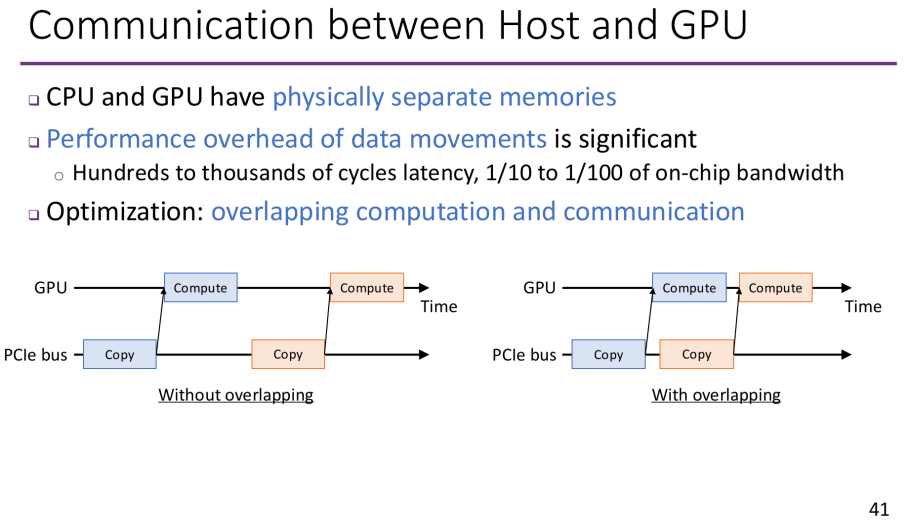


Figure 19: Overlapping host-device communication with GPU computation. Without overlap, copy and compute phases serialize. With overlap, one chunk can be computed on the GPU while another chunk is transferred over PCIe, improving pipeline utilization.

A common optimization is to divide data into chunks and pipeline communication with

computation. While the GPU computes on chunk i, the I/O bus transfers chunk $i+1$ or copies back results for chunk $i-1$. This resembles the broader I/O principle of overlapping slow external movement with useful computation.

Pinned host memory can improve GPU transfer performance because it gives the DMA engine stable physical pages. As with other DMA use, this introduces resource-management concerns: pinned memory cannot be paged out, so excessive pinning reduces OS flexibility.

11.3 CPU-GPU Roles

There are two broad ways to view CPU-GPU cooperation.

11.3.1 CPU as manager, GPU as accelerator

The CPU handles control flow, system management, I/O, scheduling decisions, memory management, and command submission. The GPU performs the application's heavy parallel computation. In this model, the CPU may block while waiting for GPU completion, or it may perform auxiliary work while the GPU runs.

This model is common when the computational kernel is massively parallel and regular: dense linear algebra, image processing, neural-network operations, physical simulation, and rendering.

11.3.2 CPU and GPU as cooperative coprocessors

In heterogeneous computing, the CPU and GPU both perform substantial parts of the application. The CPU may handle irregular control flow, small tasks, latency-sensitive logic, or sequential phases; the GPU handles data-parallel phases. Good performance requires partitioning work according to each processor's strengths and minimizing data exchange overhead.

Deep Dive

The GPU topic reinforces a recurring I/O principle: moving data is often as important as computing on data. A GPU with enormous arithmetic throughput can still underperform if each computation requires frequent host-device transfers. High-performance programs keep data resident on the device, batch work, overlap transfers with computation, and minimize synchronization.

12 Putting the Pieces Together

The mechanisms in this lecture form a layered I/O stack.

At the hardware level, devices expose registers and buffers, often through memory-mapped I/O. Buses and bridges connect different speed domains. Device controllers and DMA engines perform work independently of the CPU.

At the kernel level, device drivers translate standard abstractions into device-specific commands. The OS protects device access through virtual memory, checks permissions through system calls, schedules competing requests, handles interrupts, and coordinates buffers with virtual memory and caches.

At the application level, programmers see files, sockets, streams, block devices, asynchronous queues, and accelerator runtimes. These abstractions hide many hardware details but do not eliminate performance consequences. A program that reads one byte at a time from a disk may pay huge overhead; a program that ignores short non-blocking writes may lose data; a GPU program that repeatedly copies small buffers over PCIe may waste most of its time in communication.

Definition

The I/O abstraction boundary is the boundary between device-specific mechanisms and the uniform interfaces offered to software. A successful boundary protects hardware, hides diversity, exposes enough performance information to write efficient programs, and enables the OS to schedule and recover from errors.

13 Summary

I/O devices are essential because they connect computation to the outside world. They are difficult because they are diverse, slow relative to processors, and unpredictable. The system therefore uses specialized mechanisms rather than treating I/O as ordinary memory.

Memory-mapped I/O places device registers and buffers into the physical address space, while virtual memory protects those ranges from direct uncontrolled user access. Applications usually access devices through system calls, but special mappings can expose selected device regions to user code when appropriate. Device drivers hide vendor-specific interfaces behind standard OS abstractions such as block devices, character devices, network interfaces, and GPU runtimes.

I/O notification is handled by polling and interrupts. Polling is simple and can be efficient for frequent predictable events, but it wastes cycles or adds latency when events are rare. Interrupts let the CPU do other work until a device needs service, but they have overhead and implementation complexity. Hybrid strategies combine interrupts for wakeup with polling for batched service.

Programmed I/O makes the CPU copy every byte, which is simple but wasteful. DMA offloads bulk transfers to hardware, allowing the CPU to compute while data moves. DMA introduces important virtual-memory and cache-coherence issues: pages must be pinned, non-contiguous virtual buffers may require scatter-gather descriptors or IOMMU translation, and caches must be synchronized with device-visible memory.

Magnetic disks demonstrate how physical mechanics shape performance: seek time and rotational latency dominate small random requests, while large sequential transfers amortize those costs. Disk scheduling and caching exploit locality. SSDs eliminate mechanical latency but introduce flash-specific constraints: page reads, block erases, out-of-place writes, garbage collection, write amplification, FTL mapping, and wear leveling. GPUs show that accelerators can also be understood as I/O devices: host-device transfers, DMA, separate memories, and communication-computation overlap are central to performance.

14 Quizzes & Exercises

14.1 Short-Answer Quiz

Answer each question in two or three sentences.

1. What are the two primary challenges identified regarding I/O devices in computer systems?
2. How does memory-mapped I/O allow a processor to communicate with an I/O device?
3. What is the role of a device driver in standardizing the I/O interface?
4. Distinguish between block devices and stream devices in terms of access patterns.
5. What is the difference between blocking and asynchronous I/O interfaces?
6. Under what specific conditions is polling considered more effective than using interrupts?
7. What is the primary disadvantage of programmed I/O (PIO)?
8. How does direct memory access (DMA) improve overall system performance?
9. Why is memory pinning necessary when using DMA with physical addresses?
10. What are the three main components of mechanical disk access time?

14.2 Answer Key

1. **Two primary challenges.** The first challenge is the extreme diversity of devices, which makes a standardized interface necessary. The second challenge is that devices are often much slower than processors and can behave unpredictably, so naive waiting would waste CPU resources.
2. **Memory-mapped I/O.** Memory-mapped I/O assigns an unused range of physical addresses to a device. Loads and stores to those addresses read and write device registers or buffers, allowing software to observe status, issue commands, and transfer data.
3. **Device drivers.** A device driver is kernel-level code that exposes a standard OS-facing interface while translating operations into device-specific register accesses, commands, and timing behavior. Drivers allow applications and the OS to use many devices without knowing each device's low-level details.
4. **Block versus stream devices.** Block devices access fixed-size blocks and usually support random addressing or seeking, as disks and SSDs do. Stream devices provide sequential data without random addressing; examples include keyboards, serial ports, and many network receive paths.
5. **Blocking versus asynchronous I/O.** A blocking interface waits until data is ready or the operation has completed before returning to the caller. An asynchronous interface returns immediately, performs work in the background, and notifies the user or process when the operation completes.
6. **When polling is effective.** Polling is effective for frequent and predictable events, especially when a device has a fixed I/O rate or when the interrupt rate would be so high that interrupt overhead and latency become unacceptable.
7. **Disadvantage of PIO.** Programmed I/O consumes CPU cycles because the processor must move each byte or word between memory and the device. This is especially inefficient for large transfers or slow devices because the CPU cannot focus on useful computation.
8. **DMA performance gains.** DMA uses a hardware engine to transfer blocks of data between memory and I/O devices without CPU involvement for every byte. The CPU only sets up the transfer and handles completion, so computation and data movement can overlap.
9. **Memory pinning.** If DMA uses physical addresses, the pages involved must remain

resident and fixed in DRAM until the transfer completes. Otherwise, the OS might swap or remap a page, causing the device to read or write the wrong memory.

10. **Mechanical disk access time.** Mechanical disk access time is mainly composed of seek time, rotational latency, and data transfer time. Controller overhead and queueing delay may also contribute.

14.3 Essay Questions

Use the following prompts to develop long-form responses.

1. **The disparity of speed and data rates.** Analyze the range of I/O data rates from keyboard input to graphics output. Explain why such variation requires different architectural approaches to buffering, notification, and data transfer.
2. **Notification strategies.** Compare polling and interrupts as notification mechanisms. Explain the advantages and disadvantages of each, and propose a hybrid design that combines their benefits.
3. **The mechanics of mechanical storage.** Describe platters, surfaces, tracks, sectors, and cylinders. Explain how this physical organization leads to seek time, rotational latency, and the importance of sequential access.
4. **DMA and cache coherence.** Explain why DMA creates a version of the cache-coherence problem. Compare software cache flushing/invalidation with hardware-coherent I/O.
5. **I/O abstraction and protection.** Discuss how virtual memory, system calls, and device drivers provide both a uniform interface for programmers and a protection mechanism for hardware and kernel memory.

14.4 Additional Practice Problems

1. A disk spins at 7200 rpm. Compute the average rotational latency in milliseconds. Then explain why the average is half a rotation rather than a full rotation.
2. A program issues 4096 separate 1-byte disk reads. Explain why the performance will be much worse than one 4096-byte read, even though the same number of bytes is requested.
3. A network card receives packets at a very high rate. Explain why interrupting the CPU for every packet may be inefficient and how batching could help.
4. A DMA engine writes into a buffer that the CPU previously read. What cache operation may be needed before the CPU reads the new data? What could go wrong if it is omitted?
5. Suppose an SSD repeatedly updates the same logical block. Explain why the physical flash pages used by that logical block will change over time.
6. For the disk request sequence $2, 1, 3, 6, 2, 5$ with initial head position 5, compute the total track movement for FIFO and SSTF. Which is smaller, and what fairness issue can SSTF introduce?
7. In a GPU program, input copy takes 2 ms, computation takes 5 ms, and output copy takes 2 ms per chunk. Sketch how multiple chunks could be pipelined to overlap copy and compute.

14.5 Glossary of Key Terms

Term	Definition
Asynchronous I/O	An interface that returns control immediately after request submission, performs the operation in the background, and reports completion later.
Block device	A device, such as a disk or SSD, that accesses fixed-size blocks and can usually address locations randomly.
Blocking interface	An I/O interface that waits until data is ready or the operation is complete before returning to the caller.
Bus	A communication path that transfers data and control signals between system components, such as an I/O bus or processor-memory bus.
Cache coherence	The problem of ensuring that multiple agents, such as CPU caches and DMA-capable devices, agree on the latest value of shared data.
Cylinder	The set of aligned tracks at the same radial position across all platter surfaces in a magnetic disk.
Device driver	Kernel code that translates standard OS abstractions into low-level hardware-specific device operations.
Direct memory access (DMA)	A hardware mechanism that transfers blocks of data between memory and an I/O device without CPU copying of every byte.
Flash Translation Layer (FTL)	SSD controller software/firmware that maps logical addresses to physical flash pages and manages garbage collection, wear leveling, and block status.
Interrupt	An asynchronous external signal that causes the processor to enter an interrupt handler to service a device or event.
Memory-mapped I/O	A technique that maps device registers and buffers into the physical address space so that loads and stores can access devices.
Memory pinning	Locking virtual pages into physical DRAM so they cannot be swapped, moved, or reused during a DMA operation.
Polling	Periodically checking a device status register to determine whether the device is ready, complete, or in error.
Programmed I/O (PIO)	A transfer method in which the CPU moves each byte or word between device registers and memory using explicit loads and stores.
Rotational latency	Time spent waiting for a target sector on a rotating disk to arrive under the read/write head.
Seek time	Time required for a disk's actuator arm to move the read/write heads to the desired track.
Stream device	A device that provides sequential data without random addressing, such as a keyboard, serial port, or network receive stream.
Wear leveling	An SSD management technique that spreads erases across physical blocks to avoid premature wear-out of heavily updated locations.